



CL-2001 Data Structures Lab # 5

Objectives:

1. Circular Linked List
2. Stack
3. Infix, Prefix, Post Fix Notations

Note: Carefully read the following instructions (*Each instruction contains a weightage*)

1. There must be a block of comments at start of every question's code by students; the block should contain brief description about functionality of code.
2. Comment on every function about its functionality.
3. Use understandable name of variables.
4. Proper indentation of code is essential.
5. Make separate .cpp files for all tasks and use this format **22F-1234_Task1.cpp**.
6. First think about statement problems and then write/draw your logic on copy.
7. After copy pencil work, code the problem statement on C++ compiler.
8. Make a Microsoft Word file and paste all of your C++ code with all possible screenshots of every **task output in MS word and submit .cpp file with word file**.
9. Please submit your file in this format **22F-1234_L1**.
10. Do not submit your assignment **after the deadline**.
- 11. Do not copy code from any source otherwise you will be penalized with negative marks.**



Problem 1:

You are required to implement a program that creates a circular doubly linked list, inserts nodes at the beginning and at a specific position, and finally prints the resulting list. The allocation of memory for nodes should be done dynamically. Please follow the steps below: Create a class named `Node` with an integer data field and pointers to the previous and next nodes. Ensure that the list is circular by connecting the last node to the first node. Write a class named `CircularDoublyLinkedList` that manages the linked list. This class should have member functions for creating a new node, inserting a node at the start, inserting a node at a specific position, and printing the list.

- Write a function **`insertAtStart`** that takes a value as input and inserts a new node with that value at the beginning of the circular doubly linked list.
- Write a function **`insertAtPosition`** that takes a value and a position as input and inserts a new node with that value at the specified position.

Assume the position is 1-based. Write a function `printList` that traverses the circular doubly linked list and prints its elements. You should demonstrate the functionality of your program by creating a circular doubly linked list, inserting nodes at the start and a specific position, and then printing the resulting list. Note: Ensure proper memory management by deallocating memory after use.

Note: Only implement the highlighted functions.

```
#include <iostream>
using namespace std;
```

```
class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int value) {
        data = value;
        next = prev = nullptr;
    }
};
```

```
class CircularDoublyLinkedList {
private:
```



```
Node* head;
```

```
public:
```

```
    CircularDoublyLinkedList() {  
        head = nullptr;  
    }
```

```
    Node* createNode(int value) {  
        return new Node(value);  
    }
```

```
    void insertAtStart(int value) {
```

```
        \inset your code here
```

```
    }
```

```
    void insertAtPosition(int value, int position) {
```

```
        \inset your code here
```

```
    }
```

```
    void printList() {  
        if (head == nullptr) {  
            cout << "List is empty.\n";  
            return;  
        }  
    }
```

```
        Node* current = head;  
        do {  
            cout << current->data << " ";  
            current = current->next;  
        } while (current != head);  
        cout << endl;
```

```
    }
```

```
};
```

```
int main() {  
    CircularDoublyLinkedList list;
```

```
    list.insertAtStart(10);
```



```
list.insertAtStart(8);
list.insertAtStart(5);

cout << "Original List: ";
list.printList();

list.insertAtPosition(7, 2);

cout << "List after inserting at position 2: ";
list.printList();

return 0;
}
```

Problem 2:

Create a data structure `twoStacks` that represents two stacks. Implementation of `twoStacks` should use only one **array**, i.e., both stacks should use the same array for storing elements. Following functions must be supported by `twoStacks`.

- `push1(int x)` → pushes `x` to first stack
- `push2(int x)` → pushes `x` to second stack
- `pop1()` → pops an element from first stack and return the popped element
- `pop2()` → pops an element from second stack and return the popped element
-

Problem 3:

Implement a special stack using a singly **linked list** that supports the following operations:

- `void push(int x)`: Pushes an element onto the stack.
- `int pop()`: Removes and returns the top element of the stack. If the stack is empty, return -1.
- `int getMin()`: Returns the minimum element in the stack. If the stack is empty, return -1.

You need to design the data structure and implement the functions to achieve the desired functionality.

Problem 4:



Write a C++ program to convert infix notation to postfix notation.

1. Use the following expression to evaluate the expression. “**a+b*(c^d-e)^(f+g*h)-I**” (This notation will be passed to the program as a string)
2. Also write a code to change the following postfix notation to Infix notation. Check for the following input. **abc++**
3. Also write a C++ program to convert the following prefix notation to postfix notation. Run the code on the following input. ***+AB-CD**.
- 4.

